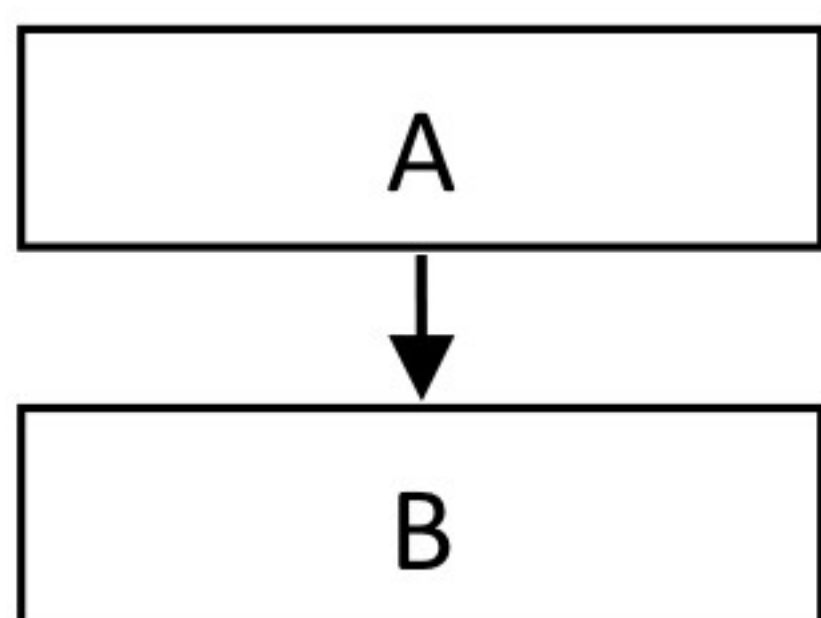


Inheritance

- Inheritance એ process છે, જેનાથી class તેના parent class ની properties અને methods ને inherit કરી શકે છે.
- Old parent class માંથી નવો child class બનાવવાની process ને inheritance કહેવામાં આવે છે.
- નવા class ને derived class અને old class ને base class કહેવામાં આવે છે.
- જ્યારે તમે existing class માંથી inherit કરો છો, ત્યારે તમે parent class ની methods and fields નો reuse કરી શકો છો, અને તમે નવી methods અને fields ને add કરી શકો છો.
- Private properties સિવાય super class ની તમામ properties તેના subclass માં extends કીવર્ડનો ઉપયોગ કરીને inherit કરી શકાય છે.

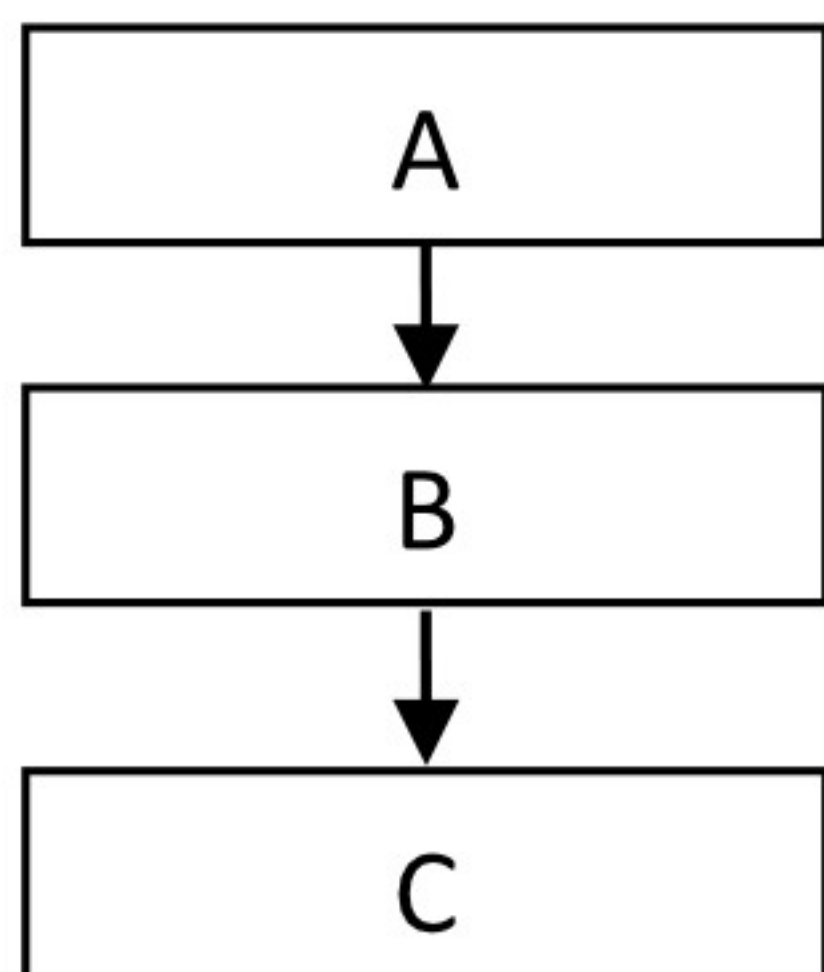
Types of Inheritance

Single Inheritance



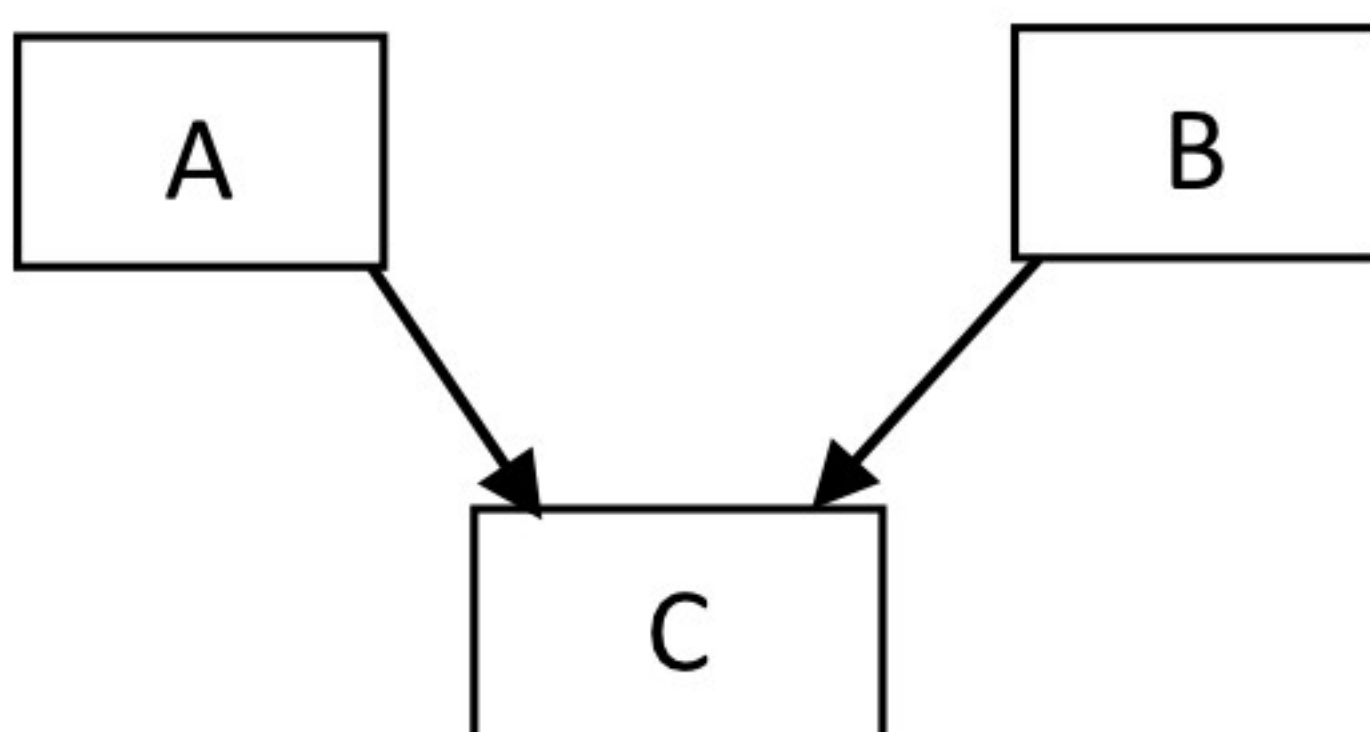
- જો કોઈ class એક જ classમાં થી derived થતો હોય તો તેને single inheritance કહેવામાં આવે છે.
- Class B એ class A માંથી derived થાય છે

Multilevel Inheritance



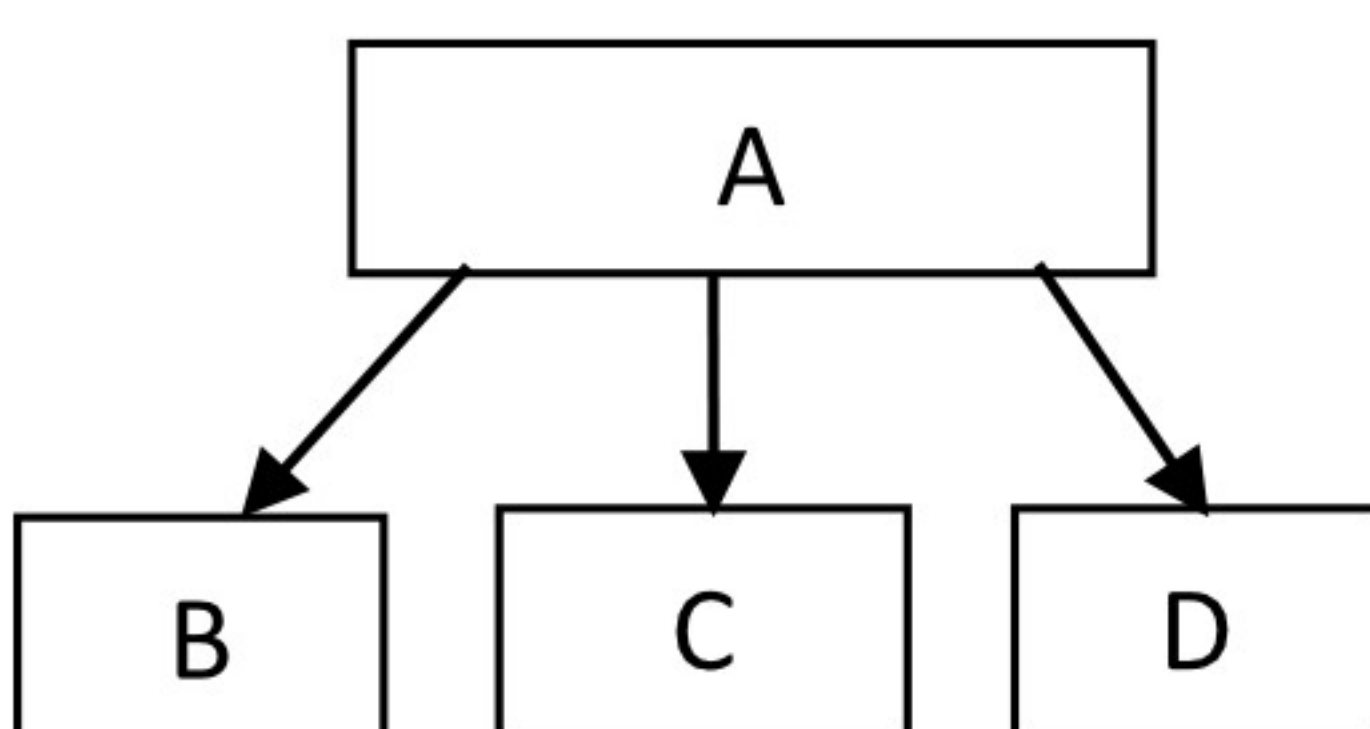
- જો class એ બીજા class માંથી derived થતો હોય અને તે class પણ બીજા class માંથી derived થતો હોય તો તેને multilevel inheritance કહેવામાં આવે છે.
- અહીં, class C એ class B માંથી derive થયો છે અને class B એ class A માંથી derived થયો છે, તેથી તેને multilevel inheritance કહેવામાં આવે છે.

Multiple Inheritance



- જો class એક કરતા વધારે class માંથી derived થાય તો તે multiple inheritance કહે છે.
- તે જાવા માં class દ્વારા સપોર્ટેડ નથી.

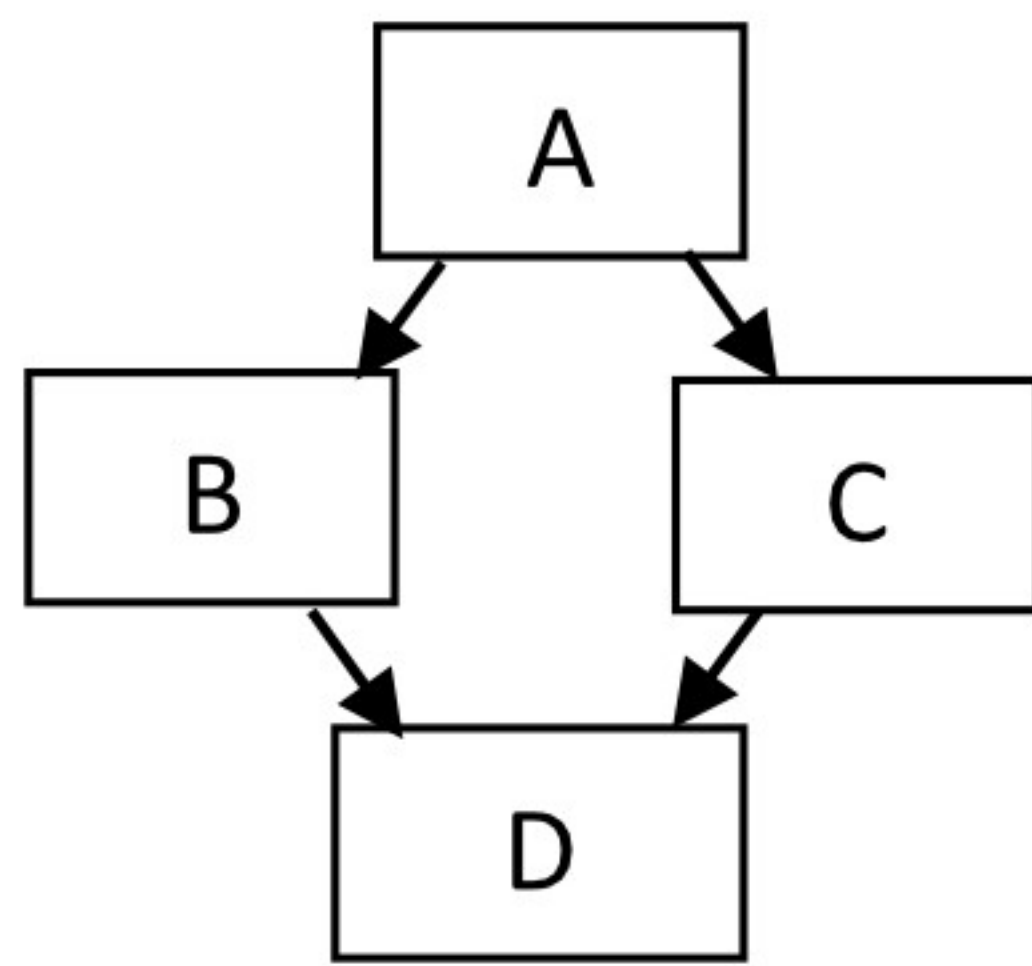
Hierarchical Inheritance



- જો એક કરતા વધુ class એક class માંથી derived કરવામાં આવે તો તેને hierarchical inheritance કહેવામાં આવે છે.
- અહીં class B, class C અને class D એ class A માંથી derived થયા છે.

Hybrid Inheritance

- Hybrid inheritance એ single અને multiple inheritanceનું combination છે.
- જાવા multiple inheritance ને support નથી કરતું, તેથી hybrid inheritance શક્ય નથી.



Example

```
class A
{
    public void displayA()
    {
        System.out.println("class A method");
    }
}
class B extends A           //Single Inheritance - class B is derived from class A
{
    public void displayB()
    {
        System.out.println("class B method");
    }
}
class C extends B // Multilevel Inheritance - class C is derived from class B
{
    public void displayC()
    {
        System.out.println("class C method");
    }
}
class D extends A //Hierarchical Inheritance - Class B and Class D are derived from Class A
{
    public void displayD()
    {
        System.out.println("class D method");
    }
}
class Trial
{
    public static void main(String []args)
    {
        B b=new B();
        C c=new C();
    }
}
```

```

        D d=new D();
        b.displayB();
        c.displayC();
        d.displayD();
    }
}

```

- Class B અને class D એ class A માંથી derive થયા છે તેથી તે Hierarchical Inheritance નું ઉદાહરણ છે.

Method Overriding (મેથડ ઓવરરાઈડિંગ)

- જ્યારે subclass માં કોઈ method declare કરવામાં આવે જેનું નામ અને parameters તેના સુપરક્લાસમાં declare કરેલ method જેવા જ હોય, ત્યારે sub class ની method super class ની method ને ઓવરરાઈડ કરે છે.
- ઓવરરાઈડિંગનો ફાયદો એ છે: subclass specific behavior ને define કરી શકાય છે. જેનો અર્થ છે કે subclass તેની જરૂરિયાતને આધારે superclass ની method ને implement કરી શકે છે.
- રનટાઈમ પોલિમોર્ફિઝમ માટે મેથડ ઓવરરાઈડિંગનો ઉપયોગ થાય છે.
- Object oriented ની language માં , ઓવરરાઈડિંગનો અર્થ કોઈપણ existing method ની functionality ને ઓવરરાઈડ કરવી એવો થાય છે.

Example:

```

class A
{
    public void display()
    {
        System.out.println("Class A");
    }
}
class B extends A
{
    public void display()
    {
        System.out.println("Class B");
    }
}
class Trial
{
    public static void main(String args[])
    {
        A a = new A();           // A reference and object
        B b = new B();           // A reference but B object
        a.display();              // Runs the method in A class
        b.display();              // Runs the method in B class
    }
}

```

Output:

Class A

Class B

- ઉપરના ઉદાહરણમાં, તમે જોઈ શકો છો કે b એ A type નો હોવા છતાં તે class B ની display () method ને call કરે છે.
- કારણ કે, કમ્પાઈલ સમયે object નો type check કરવામાં આવે છે. જો કે, રનટાઈમ એ JVM object નો type નક્કી કરે છે અને તે જ method ને call કરે જે particular object ને અનુરૂપ હોય.

Rules for method overriding

- Overridden થયેલ methods માં arguments list same હોવા જોયે.
- રીટર્ન type એ સુપરક્લાસમાં original overridden methodમાં declared કરેલા રીટર્ન type જેવો હોવો જોઈએ.
- જો subclass દ્વારા inherit કરવામાં આવે તો જ Instance methods ઓવરરાઈડ કરી શકાય છે.
- Final declared કરાયેલ method ને ઓવરરાઈડ કરી શકાતી નથી.
- Static declare કરાયેલ method ને ઓવરરાઈડ કરી શકાતી નથી પરંતુ ફરીથી declare કરી શકાય છે.
- જો કોઈ method inherit કરી શકાતી નથી, તો તેને ઓવરરાઈડ કરી શકશે નહીં.
- Same package માં subclass એ super class ની કોઈ પણ instance method જે private અને final declare કરેલ ના હોય તેને override કરી શકે છે.
- Different package માં subclass એ super class ની કોઈ પણ method જે final declare કરેલ ના હોય અને public અથવા protected declare કરેલ હોય તેને override કરી શકે છે.
- Constructors ને ઓવરરાઈડ કરી શકાતા નથી.

super keyword

- જાવામાં સુપર કીવર્ડ નો use immediate parent class ના object ને refer કરવા માટે થાય છે.

super To Access super-class Members

- Method override થાય ત્યારે overridden method ને super keyword નો use કરી ને call કરી શકાય છે

Example:

```
class A
{
    String name = "Class A";
    public void display()
    {
        System.out.println("Class A display method called..");
    }
}
class B extends A
{
    String name = "Class B";
    public void display()
    {
        System.out.println("Class B display method called..");
    }
    void printName()
    {
```

```
//this will print value of name from subclass(B)
System.out.println("Name from subclass : " + name);

// this will print value of name from superclass(A)
System.out.println("Name from Superclass: " + super.name);

//invoke display() method of Class B method
display();

// invoke display() method of class A(superclass) using super
super.display();
}
}
class SuperDemo
{
    public static void main(String args[])
    {
        B b1 = new B();
        b1.printName();
    }
}
```

Output:

Name from subclass : Class B
Name from Superclass: Class A
Class B display method called..
Class A display method called..

- અહીં, સબક્લાસની display() method તેના સુપર-ક્લાસની display() method ને ઓવરરાઈડ કરે છે.
- તેથી, સબક્લાસ (B) ની અંદર સુપરક્લાસ (A) members ને call કરવા માટે સુપર કીવર્ડ વપરાય છે.

super To Call super-class Constructor

- દરેક વખતે સબક્લાસનું પેરામીટરાઈઝડ અથવા નોન-પેરામીટરાઈઝડ કન્સ્ટ્રક્ટર બનાવવામાં આવે છે, by default સુપરક્લાસના default કન્સ્ટ્રક્ટરને implicitly call કરવામાં આવે છે.
- સુપરક્લાસ કન્સ્ટ્રક્ટરને call કરવા માટે syntax:

super();
OR
super(parameter list);
- નીચે આપેલ ઉદાહરણ બતાવે છે કે સુપરક્લાસ ના કન્સ્ટ્રક્ટરને call કરવા માટે સુપર કીવર્ડનો ઉપયોગ કેવી રીતે કરવો.

Example:

```
class A
{
    A()
    {
        System.out.println("Super class default constructor called..");
    }
    A(String s1)
```

```

    {
        System.out.println("Super class parameterized constructor called: "+s1);
    }
}
class B extends A
{
    /* Implicitly default constructor of superclass(A) will be called. Whether you define
    super or not in subclass(B) constructor */
    B()
    {
        System.out.println("Sub class default constructor called..");
    }
    /* To call a parameterized constructor of superclass(A) you must write super() with
    same number of arguments*/
    B(String s1)
    {
        super("Class A");
        System.out.println("Sub class parameterized constructor called: " + s1);
    }
}
class SuperConDemo
{
    public static void main(String args[])
    {
        B b1 = new B();
        B b2 = new B("Class B");
    }
}

```

Output:

```

Super class default constructor called..
Sub class default constructor called..
Super class parameterized constructor called: Class A
Sub class parameterized constructor called: Class B

```

- અહીં superclass નું default constructor automatic call થાય છે જ્યારે superclass ના parameterized constructor ને call કરવા માટે super keyword અને matching parameters નો use કરવામાં આવે છે.

Dynamic Method Dispatch

- Dynamic method dispatch નો use overridden method ના call ને run time એ resolve કરે છે. Compile time નહિ.
- તે રન-ટાઈમ પોલિમોર્ફિઝમ તરીકે પણ ઓળખાય છે.
- સુપરક્લાસ એ સબક્લાસ ના object ને refer કરી શકે છે. તે અપકાસ્ટિંગ તરીકે ઓળખાય છે.
- જ્યારે ઓવરરાઈડ method ને સુપરક્લાસ ના reference દ્વારા call કરવામાં આવે છે, ત્યારે કઈ method call થાશે તેનું decision reference variable કયા object ને refer કરે છે તેના પર આધાર રાખે છે.
- આ decision રનટાઈમ પર કરવામાં આવે છે.

Example:

```
class A
{
    void callme()
    {
        System.out.println("Inside A's callme method");
    }
}
class B extends A
{
    void callme()
    {
        System.out.println("Inside B's callme method");
    }
}
class C extends A
{
    void callme()
    {
        System.out.println("Inside C's callme method");
    }
}
class DispatchDemo
{
    public static void main(String args[])
    {
        A a = new A();           // object of type A
        B b = new B();           // object of type B
        C c = new C();           // object of type C
        A r;                     // obtain r a reference of type A
        r = a;                   // r refers to an A object
        r.callme();               // calls A's version of callme()
        r = b;                   // r refers to a B object
        r.callme();               // calls B's version of callme()
        r = c;                   // r refers to a C object
        r.callme();               // calls C's version of callme()
    }
}
```

Output:

```
Inside A's callme method
Inside B's callme method
Inside C's callme method
```

Object Class

- Object class (java.lang.Object) એ java માં બધા class નો super class છે

- Object class એ બધા class ને common behaviors આપે છે જેવા કે object ને convert કરવા, object ને string માં convert કરવા, object ને notified કરવા etc..
- જાવા **object class** ની **methods** નીચે આપેલ છે:

Method	Description
equals()	બે object ને compare કરવામાટે
getClass()	object ના class નું રનટાઈમ representation આપે છે. આનો ઉપયોગ કરીને આપણે class નું નામ, તેના સુપરક્લાસ વગેરે જેવી class વિશેની માહિતી મેળવી શકીએ છીએ.
toString()	object નું string representation આપે છે.
notify()	આ object's ના મોનિટરની રાહ જોતા, એક થ્રેડ ને Wake up કરે છે.
notifyAll()	આ object's ના મોનિટરની રાહ જોતા, બધા થ્રેડ ને Wake up કરે છે.
wait()	અન્ય થ્રેડ notifies ન કરે ત્યાં સુધી, current thread ને રાહ જોવાનું કહે છે.

Example:

```

class parent
{
    inti = 10;
    Integer i1 = new Integer(i);
    void PrintClassName(Object obj)    // Pass object of class as an argument
    {
        System.out.println("The Object's class name is ::" + obj.getClass().getName());
    }
}
class ObjectClassDemo
{
    public static void main(String args[])
    {
        parent a1 = new parent();
        a1.PrintClassName(a1);
        System.out.println("String representation of object i1 is:: "+a1.i1.toString());
    }
}

```

Output:

The Object's class name is :: parent
String representation of object i1 is:: 10

Packages

- જાવા પેકેજ એ same type ના classes, interfaces અને sub-packages નું group છે.
- Packages નો ઉપયોગ naming conflicts ને રોકવા માટે કરવામાં આવે છે અને access protection આપે છે.
- તેનો ઉપયોગ classes અને interfaces ના categorize માટે પણ થાય છે. જેથી તેઓ સરળતાથી maintain કરી શકાય.
- subpackage નો use કરી ને package ને categorize કરી શકાય છે . પેકેજની અંદરના પેકેજને subpackage કહેવામાં આવે છે.
- પેકેજને બે form માં categorized કરી શકાય છે: built-in package અને user-defined package.

- **built-in packages** : હાલના જાવા પેકેજ જેમ કે **java.lang, java.util, java.io, java.net, java.awt.**
- **User-defined-package** : classes અને interface ને categorize કરવા માટે user દ્વારા બનાવેલ જાવા પેકેજ.
- Programmers પોતાના classes અને interfaces નું પોતાનું package બનાવી શકે છે.

Creating a package

- પેકેજ બનાવવા માટે, package statement પછી package નું name આપવાનું હોય છે.
- package statement એ source file માં પ્રથમ લીટી હોવું જોઈએ. દરેક source file માં ફક્ત એક જ પેકેજ સ્ટેટમેન્ટ હોઈ શકે છે.
- જો પેકેજ સ્ટેટમેન્ટનો ઉપયોગ ન કરવામાં આવે તો class, interface વગેરે unnamed package માં મૂકવામાં આવશે.

Example:

```
package mypack;
class Book
{
    String bookname;
    String author;
    Book()
    {
        bookname = "Complete Reference";
        author = "Herbert";
    }
    void show()
    {
        System.out.println("Book name is :: "+bookname+"\nand author name is :: "+
author);
    }
}
class DemoPackage
{
    public static void main(String[] args)
    {
        Book b1 = new Book();
        b1.show();
    }
}
```

Compile: javac -d . DemoPackage.java

To run the program : java mypack.DemoPackage

Output:

Book name is :: Complete Reference
and author name is :: Herbert

- -d એ સ્વીચ છે જે કમ્પાઈલરને ક્લાસ ફાઈલ ક્યાં મૂકવી તે કહે છે. જેમ કે, /home (Linux), d:/abc (windows) etc.
- . (ડોટ) current folder ને represent કરે છે.

Import Package

- import keyword નો ઉપયોગ java source file માં built-in અને user-defined packages ને import કરવા માટે થાય છે.
 - જો class same પેકેજમાં બીજો class use કરવા માંગે છે, તો પેકેજ import કરવાની જરૂર નથી.
 - પરંતુ, જો કોઈ class બીજા class નો use કરવા માંગે છે જે એક જ પેકેજમાં નથી, તો import કીવર્ડનો ઉપયોગ તે પેકેજને તમારી java source file import કરવા માટે કરવામાં આવે છે.
 - class માં એક કરતા વધારે import statements હોય શકે છે.
 - પેકેજની બહારથી access કરવાની ત્રણ રીતો છે:
 - 1) import package.*;
 - 2) import package.classname;
 - 3) fully qualified name
- 1) **Using package.*:**
 - જો તમે package.* નો use કરો છો. તો પછી આ પેકેજના બધા classes અને interfaces એક્સેસ કરી શકાય પરંતુ subpackages એક્સેસ થશે નહિ
Syntax: import package.*;
 - 2) **Using package.classname :**
 - જો તમે package.classname નો ઉપયોગ કરો છો તો આ પેકેજનો ફક્ત declared class જ access થશે.
Syntax: import package.classname;
 - 3) **Using fully qualified name :**
 - જો તમે fully qualified name નો ઉપયોગ કરો છો તો આ પેકેજનો ફક્ત declared class જ access થશે. તેથી, પેકેજ import કરવાની જરૂર નથી.
 - પરંતુ, જ્યારે તમે class અને interface ને access કરો ત્યારે તમારે દર વખતે fully qualified name વાપરવાની જરૂર છે.
 - સામાન્ય રીતે જ્યારે બે પેકેજોના નામ same હોય છે ત્યારે તેનો ઉપયોગ થાય છે દા.ત. java.util અને java.sql પેકેજોમાં Date ક્લાસ હોય છે.

Example:

File 1: **A.java**

```
package pack;
public class A
{
    public void display()
    {
        System.out.println("Welcome to package pack...");
    }
}
```

File 2: **Q.java**

```
package pack;
public class Q
{
    public void Q_display()
    {
        System.out.println("Welcome to package pack through qualified
name...");
    }
}
```

```
}
```

File 3: **B.java**

```
package mypack;
import pack.*;           // Here, you can also use import pack.A ;
class B
{
    public static void main(String args[])
    {
        A a1 = new A();
        pack.Q q1= new pack.Q();
        /*Here Q is class in package A. If you want to access Q_display() method of class Q
        using fully qualified name then no need to import package named pack */
        a1.display();
        q1.Q_display();
    }
}
```

Output: java mypack.B

Welcome to package pack...

Welcome to package pack through qualified name...

- ઉપરના ઉદાહરણમાં, class A, class Q અને તેની બધી methods ને public તરીકે declared કરવી આવશ્યક છે નહીં તો તેઓ class Bમાં access થશે નહિ.

Visibility and Access Rights

class \ have access to	Private	Default	Protected	Public
own class	yes	yes	yes	yes
subclass - same package	no	yes	yes	yes
subclass - another package	no	no	yes	yes
class - another package	no	no	no	yes

Interfaces

- ઇન્ટરફેસ એ abstract methods નું collection છે. ઇન્ટરફેસ એ class નથી.
- જ્યારે તમે ઇન્ટરફેસ બનાવો છો ત્યારે તે class કેવી રીતે કરશે તેના વિશે કંઈ પણ કહ્યા વિના class શું કરી શકે છે તે defines કરે છે.
- ઇન્ટરફેસમાં ફક્ત static constants and abstract methods ઓ હોય છે.
- જાવામાં ઇન્ટરફેસ એ fully abstraction આપે છે. જાવા ઇન્ટરફેસમાં abstract methods ઓ હોઈ શકે છે.
- By default (Implicitly) ઇન્ટરફેસ એબ્સ્ટ્રેક્ટ હોય છે, ઇન્ટરફેસ માં fields(data members) એ public, static અને final હોય છે અને methods public અને abstract હોય છે.
- તેનો ઉપયોગ જાવામાં fully abstraction અને multiple inheritance કરવા માટે થાય છે.
- Class અને ઇન્ટરફેસ વચ્ચે similarity નીચે આપેલ છે:
 - ઇન્ટરફેસમાં ઘણી બધી methods હોઈ શકે છે
 - ઇન્ટરફેસ ને .java ના extension થી store કરવામાં આવે છે. ઇન્ટરફેસ file નું નામ અને ઇન્ટરફેસ નું નામ સરખા હોય છે.

- ઇન્ટરફેસનો bytecode ને .class ફાઈલમાં save કરવામાં આવે છે.
- class અને ઇન્ટરફેસ વચ્ચેનો તફાવત નીચે આપેલ છે:

Class	Interface
● તમે class instantiate કરી શકો છો.	● તમે ઇન્ટરફેસ instantiate કરી શકતા નથી.
● તેમાં ડિફોલ્ટ તેમજ પેરામીટરાઈઝડ કન્સ્ટ્રક્ટર છે.	● તેમાં કોઈપણ constructors નથી.
● બધી methods ની definition હોવી જોઈએ અથવા methodsને abstract તરીકે declare કરો.	● ઇન્ટરફેસમાં બધી method એ by default રીતે abstract હોય છે.
● બધા variables by default રીતે instance છે.	● બધા variables by default રીતે static અને final હોય છે, અને definition સમયે value આપવી જરૂરી છે.
● class ફક્ત એક જ class ને inherit કરી શકે છે અને ઘણા ઇન્ટરફેસ ને implement કરી શકે છે.	● ઇન્ટરફેસ કોઈપણ class ને inherit કરી શકે નહિ જ્યારે તે ઘણા ઇન્ટરફેસને extend કરી શકે.

Declaring Interfaces

- ઇન્ટરફેસ કીવર્ડનો ઉપયોગ ઇન્ટરફેસ declare કરવા માટે થાય છે.

Syntax: NameOfInterface.java

```
import java.lang.*;
public interface NameOfInterface
{
    //Any number of final, static fields
    //Any number of abstract method declarations
}
```

Example: DemoInterface.java

```
interface DemoInterface
{
    int i = 10;
    void demo();
}
```

- ઉપરના ઉદાહરણમાં, ઇન્ટરફેસનું નામ DemoInterface છે અને તેમાં integer typeનો variable i અને demo() નામની abstract method છે.

Implementing Interfaces

- class ઇન્ટરફેસને implement કરવા માટે implements કીવર્ડનો ઉપયોગ થાય છે.
- class એ ઇન્ટરફેસ ને implements કરે છે તેનો અર્થ class એ contract પર sign કરે છે જે ઇન્ટરફેસ નું specific behaviors ને perform કરવા agree હોય છે.
- જો કોઈ class એ interface ના બધા behaviors ને perform કરતો નથી, તો class પોતાને abstract તરીકે declare કરે છે.

Example: DemoInterfaceImp.java

```
public class DemoInterfaceImp implements DemoInterface
{
    public void demo()
    {
        System.out.println("Value of i is :: "+i);
    }
}
```

```
}  
public static void main(String args[])  
{  
    DemoInterfaceImp d = new DemoInterfaceImp();  
    d.demo();  
}  
}
```

Output:

Value of i is :: 10

Inheritance on Interfaces

- આપણે બધા જાણીએ છીએ કે એક class બીજા class ને extend કરી શકે છે. એ જ રીતે ઇન્ટરફેસ બીજા ઇન્ટરફેસને extend કરી શકે છે.
- Extends કીવર્ડનો ઉપયોગ ઇન્ટરફેસને extends કરવા માટે થાય છે, અને ચાઇલ્ડ ઇન્ટરફેસ પેરન્ટ ઇન્ટરફેસની methods ને inherit કરે છે.

Example:

```
public interface A  
{  
    void getdata(String name);  
}  
public interface B extends A  
{  
    void setdata();  
}  
class InheritInterface implements B  
{  
    String display;  
    public void getdata(String name)  
    {  
        display = name;  
    }  
    public void setdata()  
    {  
        System.out.println(display);  
    }  
    public static void main(String args[])  
    {  
        InheritInterface obj = new InheritInterface();  
        obj.getdata("Welcome TO Heaven");  
        obj.setdata();  
    }  
}
```

Output:

Welcome TO Heaven

- ઇન્ટરફેસ B ની એક method છે, પરંતુ તે ઇન્ટરફેસ A માંથી inherits થાય છે; આમ, B ને implements કરતો class InheritInterface બે method ને implement કરવાની જરૂર છે.

Multiple Inheritance using Interface

- જો class મલ્ટીપલ interfaces ને implements કરે છે, અથવા ઇન્ટરફેસ મલ્ટીપલ ઇન્ટરફેસને extends કરે છે તો આને multiple inheritance તરીકે ઓળખાય છે.
- class ફક્ત એક parent class ને extend કરી શકે છે. Multiple inheritances allowed નથી. જો કે, ઇન્ટરફેસ એકથી વધુ પેરન્ટ ઇન્ટરફેસને extend કરી શકે છે.
- extends કીવર્ડનો ઉપયોગ એકવાર થાય છે, અને parent interfaces ને (,) થી separated list declared કરવામાં આવે છે.

Example:

```
public interface A
{
    void getdata(String name);
}
public interface B /* Here we can also extends multiple interface like interface B
extends A,C*/
{
    void setdata();
}
class InheritInterface implements A, B
{
    String display;
    public void getdata(String name)
    {
        display = name;
    }
    public void setdata()
    {
        System.out.println(display);
    }
    public static void main(String args[])
    {
        InheritInterface obj = new InheritInterface();
        obj.getdata("Welcome TO Heaven");
        obj.setdata();
    }
}
```

Output:

Welcome TO Heaven

Abstract Class

- class જેને abstract keyword સાથે declared કરવામાં આવે છે, તે class ને એબ્સ્ટ્રેક્ટ class તરીકે ઓળખાય છે. તેમાં abstract અને non-abstract methods (method with body) હોઈ શકે છે.

- તેને extended કરવાની અને તેની method ને implement કરવાની જરૂર છે. તેને instantiated કરી શકાતું નથી એટલે આપણે તેનો object બનાવી શકીએ નહીં.
- કોઈપણ class કે જે abstract class ને extends કરે છે તેણે સુપર class દ્વારા declared કરેલી બધી abstract methods ને define કરવી જરૂરી છે.

Syntax:

```
abstract class class_name
{
    //Number of abstract as well as non-abstract methods.
}
```

- method જેને abstract તરીકે declared કરવામાં આવે છે અને તેમાં implementation નથી, તે abstract method તરીકે ઓળખાય છે.
- method ની body તેના subclass દ્વારા defined કરવામાં આવશે. Abstract method ક્યારેય final અને static હોઈ શકતી નથી.

Syntax:

```
abstract return_type function_name ();    // No definition
```

Example:

```
abstract class A
{
    abstract void abs_method();
    public void display()
    {
        System.out.println("This is concrete method..");
    }
}
class DemoAbstract extends A
{
    void abs_method()
    {
        System.out.println("This is an abstract method..");
    }
    public static void main(String[] args)
    {
        DemoAbstract abs = new DemoAbstract();
        abs.abs_method();
        abs.display();
    }
}
```

Output:

```
This is an abstract method..
This is concrete method..
```

Final Keyword

- જાવામાં final કીવર્ડનો ઉપયોગ user ને restrict કરવા માટે થાય છે. જાવા final કીવર્ડ આની સાથે ઉપયોગ કરી શકાય છે:
 - 1) variable
 - 2) method
 - 3) class

- **Final Variable:** જો તમે કોઈપણ variable ને final તરીકે declare કરો છો, તો તમે તે final variable નું value બદલી શકતા નથી (It will be constant).
 - Variable કે જે final તરીકે declared કરવામાં આવે છે અને initialized નથી, તેને blank final variable કહેવામાં આવે છે. એક blank final variable કન્સ્ટ્રક્ટરોને તેને initialize કરવા માટે forces કરે છે.
- **Final Method:** final તરીકે declared કરાયેલ Methods ને ઓવરરાઈડ કરી શકાતી નથી.
- **Final Class:** જાવા classes ને final તરીકે declared કરી શકાતા નથી, કારણ કે તેનો inherit કરી શકાતો નથી.
- જો તમે કોઈપણ parameter ને final તરીકે declare કરો છો, તો તમે તેની value બદલી શકતા નથી.

Example:

```
class DemoBase
{
    final inti = 1;          //final variable must be initialize.
    final void display()
    {
        System.out.println("Value of i is :: "+ i);
    }
}
class DemoFinal extends DemoBase
{
    /*void display()        // Compilation error final method cannot override.
    {
        System.out.println("Value of i is :: "+ i);
    }*/
    public static void main(String args[])
    {
        DemoFinal obj = new DemoFinal();
        obj.display();
    }
}
```

Output:

Value of iis :: 1

Difference between Method Overloading and overriding

Overriding	Overloading
• argument list ઓવરરાઈડ method ની exactly match થતી હોવી જોઈએ.	• Overloaded methods માં argument list ને બદલવી જરૂરી છે.
• રીટર્ન type સુપર ક્લાસમાં ઓવરરાઈડ method જેવો જ હોવો જોઈએ.	• ઓવરલોડ methods રીટર્ન type બદલી શકે છે.
• Private અને final methods ઓવરરાઈડ કરી શકાતી	• Private અને final methods ઓવરલોડ થઈ શકે છે

નથી.	
મેથડ ઓવરરાઈડિંગ એ બે classes માં થાય છે જેમાં inheritance હોય છે.	Method overloading એ class માં કરવામાં આવે છે.
Dynamic binding એ overridden methods માટે ઉપયોગમાં લેવાય છે.	Static binding એ overloaded methods માટે ઉપયોગમાં લેવાય છે.
મેથડ ઓવરરાઈડિંગ એ રન ટાઈમ પોલિમોર્ફિઝમનું ઉદાહરણ છે.	મેથડ ઓવરલોડિંગ એ કમ્પાઈલ ટાઈમ પોલિમોર્ફિઝમનું ઉદાહરણ છે.

Difference between Interface and Abstract class

Interface	Abstract class
ઈન્ટરફેસમાં ફક્ત abstract methods જ હોઈ શકે છે.	એબ્સ્ટ્રેક્ટ class માં abstract અને non-abstract methods હોઈ શકે છે.
ઈન્ટરફેસ એ multiple inheritance ને supports કરે છે.	Abstract class એ multiple inheritance ને support કરતો નથી.
ઈન્ટરફેસમાં static methods, main method અને constructor હોઈ શકે નથી.	Abstract class માં static methods, main method અને constructor હોઈ શકે.
ઈન્ટરફેસ એ abstract class નું implementation કરી શકતું નથી.	એબ્સ્ટ્રેક્ટ class એ ઈન્ટરફેસ નું implementation કરી શકે છે.